

Cluster-Saphir

Mauricio Santibañez

January 2026

1. Acceso al clúster SAPHIR desde la terminal

1.1. Conexión al VPN SAPHIR

Si el acceso se realiza desde fuera de la red interna, primero se debe activar el VPN:

```
openvpn3 session-start --config saphir-cluster
```

Para verificar que el VPN esté activo:

```
openvpn3 sessions-list
```

1.2. Acceso al clúster vía SSH

Una vez conectado al VPN (o desde la red interna), se accede al clúster usando SSH:

```
ssh santibanez@ui.institutosaphir.cl
```

Al ejecutar este comando:

- Se solicitará la contraseña asociada a la cuenta.
- Tras una autenticación exitosa, se ingresará al servidor de interfaz de usuario (UI).

1.3. Salir del clúster

Para cerrar la sesión SSH de forma segura y salir del clúster:

```
exit
```

Alternativamente, se puede usar:

```
logout
```

o la combinación de teclas:

```
Ctrl + D
```

1.4. Desconexión del VPN

Una vez fuera del clúster, se recomienda cerrar el VPN:

```
openvpn3 session-manage --config saphir-cluster --disconnect
```

Para confirmar que no hay sesiones activas:

```
openvpn3 sessions-list
```

Si no se muestra ninguna sesión, el VPN ha sido desconectado correctamente.

2. Programas y paquetes disponibles en el clúster

El clúster SAPHIR cuenta con una amplia variedad de programas científicos y herramientas de desarrollo preinstaladas, orientadas principalmente a física de partículas, astrofísica, simulaciones Monte Carlo y análisis de datos. Algunos paquetes requieren una configuración previa del entorno antes de poder ser utilizados.

2.1. Comando use y gestión de entornos

Muchos programas se activan mediante el comando `use`, el cual configura automáticamente las variables de entorno necesarias.

Para listar todos los paquetes disponibles:

```
use
```

Para activar un paquete específico:

```
use nombre_del_paquete
```

Ejemplo:

```
use anaconda
```

2.2. Paquetes disponibles más comunes

Comando	Paquete
use anaconda	Distribución Python Anaconda
use corsika	Simulación de cascadas de partículas
use cuda	CUDA Toolkit (GPU)
use fluka	Paquete de simulación FLUKA
use gcc12	GCC versión 12
use gcc13	GCC versión 13
use geant4	Toolkit Geant4
use genie	Generador de eventos GENIE
use mg5	MadGraph5
use mpich	MPI (MPICH)
use openmpi	MPI (OpenMPI)
use pythia6	Pythia 6
use pythia	Pythia 8
use xilinx	Software Xilinx

2.3. CernVM-FS (CVMFS)

El clúster tiene acceso a software distribuido a través de **CernVM-FS**, un sistema de archivos de solo lectura montado en:

```
/cvmfs
```

Los datos se descargan automáticamente la primera vez y luego se usan desde caché local.
Repositorios disponibles:

- `/cvmfs/atlas.cern.ch` – Software ATLAS
- `/cvmfs/atlas-condb.cern.ch` – Datos de condiciones ATLAS
- `/cvmfs/atlas-nightlies.cern.ch` – Builds nocturnas ATLAS
- `/cvmfs/grid.cern.ch` – LCG
- `/cvmfs/sft.cern.ch` – Externos LCG
- `/cvmfs/sft-nightlies.cern.ch` – Nightlies LCG
- `/cvmfs/unpacked.cern.ch` – Imágenes de contenedores

2.4. Uso de software ATLAS

La mayoría del software ATLAS está compilado para sistemas antiguos, por lo que se ejecuta mediante contenedores.

Para configurar el entorno ATLAS usando CentOS 7:

```
setupATLAS -c centos7
```

Contenedores disponibles incluyen: `sl6`, `sl7`, `slc6`, `centos7`, `rhel6`, `rhel7`.

2.5. CORSIKA

Para activar el entorno de CORSIKA:

```
use corsika
```

Ejecutar el programa principal:

```
c8_air_shower
```

Ejecutar ejemplos incluidos:

```
known_particles
```

Uso desde Python:

```
python
>>> import corsika
```

2.6. POWHEG-BOX

Para activar el entorno POWHEG (ejemplo proceso `hvq`):

```
use powheg-hvq
```

Generación de eventos:

```
pwhg_main
```

Esto produce un archivo de eventos en formato LHE:

```
pwgevents.lhe
```

Simulación posterior con PYTHIA:

```
main-PYTHIA-lhef
```

2.6.1. Ejecución paralela con `run_parallel`

Para ejecutar múltiples generadores en paralelo:

```
run_parallel pwhg_main 4
```

Esto genera múltiples archivos `pwgevents-XXXX.lhe`.

Simulación PYTHIA en paralelo:

```
run_pythia main-PYTHIA82-lhef-LHCb-bottom 4
```

2.7. NA64

Activar el entorno NA64:

```
use na64
```

Ejecutar COOOL sobre datos NA64:

```
cool.sh /data/na64/demo/archivo.dat
```

Ejemplo de reconstrucción:

```
example.exe /data/na64/demo/archivo.dat
```

2.8. Python y Anaconda

Activar Python Anaconda:

```
use anaconda
```

Listar módulos instalados:

```
conda list
```

Activar entorno GPU:

```
source activate gpu
```

Desactivar entorno:

```
conda deactivate
```

Instalar módulos localmente:

```
pip install nombre_modulo --user
```

2.9. ROOT

Usar ROOT desde Python:

```
python  
>>> import ROOT
```

Usar una versión específica de ROOT:

```
unset ROOTSYS  
source /opt/software/root/6.32.02/bin/thisroot.sh
```

3. Almacenamiento de datos

El clúster computacional proporciona distintas áreas de almacenamiento, diseñadas para diferentes volúmenes de datos y tipos de uso. Es fundamental utilizar cada una de ellas de manera adecuada para asegurar un uso eficiente del sistema.

3.1. Directorios de usuario

Cada usuario dispone de un directorio personal ubicado en:

`/user`

En este directorio se pueden almacenar datos de hasta aproximadamente **1 TB**. Este espacio está destinado principalmente a:

- Código fuente
- Scripts de análisis
- Resultados intermedios
- Archivos personales del usuario

3.2. Áreas de almacenamiento de gran tamaño

Para almacenar grandes volúmenes de datos (hasta decenas de TB), existen áreas de disco especiales. Un ejemplo es el almacenamiento dedicado al experimento NA64:

`/data/na64`

El acceso a estas áreas requiere permisos especiales y debe gestionarse mediante una solicitud al soporte del clúster.

3.3. Cuotas de almacenamiento

El espacio en disco disponible para cada usuario es limitado. Para verificar el uso del espacio y las cuotas asignadas, se puede utilizar el comando:

`quota`

Ejemplo de salida:

Storage quotas for user kuleshov (uid 1001):

Filesystem		blocks	quota	files	quota
pool1/user	/user	108	1073741824	24	0
pool1	/data/na64	13646415436	0	15121	0

El valor 0 indica que no existe un límite impuesto (por ejemplo, en el número de archivos).

Para visualizar la información en un formato legible por humanos, se puede usar:

`quota -s`

Ejemplo:

Storage quotas for user kuleshov (uid 1001):

Filesystem		space	quota	files	quota
pool1/user	/user	108K	1T	24	-
pool1	/data/na64	12.7T	-	14.8K	-

3.4. Buenas prácticas de almacenamiento

Los sistemas de almacenamiento en red funcionan de manera eficiente únicamente cuando los datos están organizados en archivos grandes. Se recomienda:

- Evitar guardar archivos pequeños (menores a 1 MB).
- Evitar crear demasiados archivos en un mismo directorio (más de 16 000).

- Mantener los archivos pequeños empaquetados (por ejemplo, en archivos `.tar`).
- Realizar el procesamiento de datos pequeños en discos locales.
- Utilizar el almacenamiento en red solo para datos distribuidos y de gran tamaño.

Estas prácticas mejoran significativamente el rendimiento del procesamiento de datos.

4. Acceso remoto a los datos

Además del acceso directo desde el clúster, los usuarios pueden acceder a los datos de forma remota a través del servidor de la nube del clúster.

El acceso puede realizarse mediante:

- Interfaz web
- Aplicaciones de escritorio (Linux, MacOS, Windows)
- Aplicaciones móviles (iOS y Android)

Algunas áreas del disco del clúster pueden montarse como unidades de red mediante los protocolos **SMB** o **WebDAV**. Para ello, se debe solicitar una contraseña especial al soporte del clúster.

4.1. Unidades SMB disponibles

El acceso SMB es posible únicamente desde equipos conectados a la red SAPHIR o mediante VPN (OpenVPN o FortiClient).

Unidad SMB	Descripción	Acceso
<code>//cluster.institutosaphir.cl/homes</code>	Directorios de usuarios	Lectura/escritura propios
<code>//cluster.institutosaphir.cl/shared</code>	Datos compartidos	Solo lectura
<code>//cluster.institutosaphir.cl/public</code>	Datos públicos	Solo lectura
<code>//cluster.institutosaphir.cl/internal</code>	Datos internos	Acceso restringido

4.2. Unidades WebDAV disponibles

Unidad WebDAV	Descripción	Acceso
<code>https://cluster.institutosaphir.cl/shared</code>	Datos compartidos	Lectura/escritura
<code>https://cluster.institutosaphir.cl/public</code>	Datos públicos	Solo lectura

Advertencia: A pesar de los nombres similares, las unidades SMB y WebDAV contienen datos distintos. Además, debido a las características del protocolo WebDAV, los usuarios pueden modificar todos los datos a los que tengan acceso de escritura. Se recomienda extrema precaución al utilizar este método.

5. Acceso a los datos desde Linux

En sistemas Linux, se puede acceder a las unidades de red mediante el gestor de archivos GNOME Files:

1. Seleccionar **Other Locations**.
2. En **Connect to Server**, ingresar la dirección correspondiente.

Para direcciones SMB, se debe utilizar el prefijo **smb:** y el símbolo **/** en lugar de ****.
Para direcciones WebDAV, se debe usar el prefijo **davs:** en lugar de **https:**.

5.1. Acceso mediante línea de comandos

Para acceder vía SMB desde la terminal:

```
smbclient -U username '\\cluster.institutosaphir.cl\homes'
```

o bien:

```
smbclient -U username //cluster.institutosaphir.cl/homes
```

Para acceder mediante WebDAV, se puede usar el programa **cadaver**:

```
cadaver https://cluster.institutosaphir.cl/shared
```

Las operaciones en ambos programas son similares. Para más información, consulte:

```
man smbclient  
man cadaver
```

6. Sistema por lotes (Batch System)

Para ejecutar programas que requieren grandes cantidades de tiempo de CPU o hardware especial (por ejemplo, GPUs), se debe utilizar el sistema por lotes del clúster. Este sistema permite enviar trabajos que serán ejecutados automáticamente en los nodos de cómputo disponibles.

El flujo general de ejecución es el siguiente:

UI → Batch System → Worker Nodes → Batch System → UI

En lugar de ejecutar programas directamente desde la consola, el usuario debe crear un **script de trabajo** y enviarlo al sistema por lotes, el cual se encarga de asignar los recursos y gestionar la ejecución.

6.1. Slurm Workload Manager

El clúster utiliza **Slurm Workload Manager** para la gestión de trabajos. Los trabajos se envían mediante el comando `sbatch`.

Los requisitos de recursos (tiempo, memoria, núcleos, etc.) se especifican usando directivas que comienzan con `#SBATCH`. Estas líneas son comentarios para el shell, pero son interpretadas por Slurm.

Ejemplo para solicitar dos horas de tiempo de ejecución:

```
#SBATCH --time=2:00:00
```

El trabajo se envía con:

```
sbatch script.sh
```

Las opciones indicadas en la línea de comandos sobrescriben las del script.

6.2. Trabajos simples

Ejemplo de un script básico `demo.sh`:

```
#!/usr/bin/bash
echo "Hello from the shell script!"
echo "I'm running at host $(hostname)"
```

Ejecución directa en la máquina UI:

```
sh demo.sh
```

Envío al sistema por lotes:

```
sbatch demo.sh
```

Slurm devolverá un identificador de trabajo:

```
Submitted batch job 7656
```

6.3. Monitoreo y control de trabajos

Para ver el estado del trabajo:

```
sacct -j 7656
```

Estados comunes:

- PENDING: esperando recursos
- RUNNING: en ejecución
- COMPLETED: finalizado correctamente
- CANCELLED: cancelado por el usuario

Para ver la cola de trabajos:

```
squeue
```

Información detallada de un trabajo:

```
scontrol show job=7656
```

Cancelar un trabajo:

```
scancel 7656
```

6.4. Archivos de salida

Por defecto, la salida estándar y de error se redirigen a un archivo:

```
slurm-<JobID>.out
```

Ejemplo:

```
cat slurm-7656.out
```

6.5. Scripts con requisitos explícitos

Ejemplo de script más completo `pi.sh`:

```
#!/bin/bash
#SBATCH -J Pi
#SBATCH -o pi-%j.out
#SBATCH -e pi-%j.err
#SBATCH --time=1:00:00
#SBATCH --mem=6M
```

```
./pi $*
```

Aquí:

- `-J`: nombre del trabajo
- `-o`: archivo de salida estándar
- `-e`: archivo de error
- `-mem`: memoria requerida

6.6. Reporte de eficiencia

Resumen del uso de recursos:

```
saff 7659
```

Este comando muestra:

- Tiempo real de ejecución
- Uso de CPU
- Eficiencia de memoria

Solicitar solo los recursos necesarios mejora la prioridad del trabajo en la cola.

6.7. Uso de almacenamiento temporal (TMPDIR)

Durante la ejecución, Slurm proporciona un directorio local rápido indicado por la variable:

```
$TMPDIR
```

Este espacio (hasta ~100 GB) se recomienda para:

- Archivos temporales
- Procesamiento intensivo de datos
- Reducir accesos a discos en red

Los archivos deben copiarse explícitamente desde y hacia este directorio dentro del script.

6.8. Programas paralelos con OpenMP

Para solicitar múltiples núcleos:

```
#SBATCH -c 4
```

El número de hilos también puede controlarse con:

```
export OMP_NUM_THREADS=4
```

OpenMP utiliza paralelismo por hilos dentro de un solo nodo.

6.9. Programas paralelos con MPI

MPI permite paralelismo entre múltiples nodos mediante procesos independientes.

Configuración del entorno:

```
use openmpi
```

Compilación:

```
mpicc -o pi_mpi pi_mpi.c
```

Script de envío:

```
#!/bin/bash
#SBATCH -n 4
use openmpi
mpiexec ./pi_mpi
```

Aquí:

- -n: número de procesos MPI
- Cada proceso usa un núcleo

6.10. Notas sobre escalabilidad

Un mayor número de núcleos o procesos no siempre implica mayor eficiencia. Antes de ejecuciones masivas, se recomienda:

- Realizar pruebas de escalabilidad
- Determinar el número óptimo de núcleos
- Priorizar eficiencia sobre paralelismo máximo